RentLoop – Sistem preporuke (Recommender System)

RentLoop je platforma za iznajmljivanje nekretnina koja korisnicima omogućava pretraživanje i rezervaciju smještaja na osnovu lokacije, cijene, tipa najma i drugih kriterija. Kako broj oglasa (listing-a) i korisnika raste, postaje sve teže ručno pronaći najrelevantnije opcije. Zbog toga je unutar sistema implementiran recommender mehanizam koji personalizuje sadržaj i korisniku predlaže **preporučene listing-e** ("Recommended") te omogućava **fallback prikaz popularnih listing-a** ("Popular") kada nema dovoljno signala o ponašanju korisnika.

U RentLoop aplikaciji implementirane su dvije povezane funkcionalnosti:

1. **Personalizovana preporuka listing-a** (/recommended)
2. **Popularni listing-i (fallback / cold start)** (/popular)

Obje funkcionalnosti su implementirane kao backend endpointi u okviru ListingsController i bazirane su na **hibridnom rule-based rangiranju** (bez ML.NET treniranja modela).

---

1) Personalizovana preporuka listing-a (Recommended)

Opis pristupa

Preporuke se generišu kombinovanjem:

- **signala iz zadnje korisnikove pretrage** (SearchHistory)
- **signala iz historije ponašanja** (zadnji pregledi i rezervacije)
- **popularnosti listing-a** u sistemu (views u 7 dana + rezervacije u 30 dana)
- **kvaliteta listing-a** (prosječna ocjena i broj recenzija)

Sistem zatim pravi listu kandidata (newest limit) i rangira ih prema ukupnom Score rezultatu, vraćajući korisniku **Top-N preporuka**.

1.1. Prikupljanje korisničkih signala

Za autentifikovanog korisnika prikupljaju se sljedeći podaci:

A) Zadnja pretraga korisnika (SearchHistory)

- uzima se posljednji zapis iz SearchHistory prema SearchedAt

- signali koji se koriste: CityId, RentTypeId, MinPrice, MaxPrice, RoomsCount, Guests, Sort

B) Zadnjih 10 pregleda (ListingViews)

- uzima se do 10 različitih listing-a koje je korisnik zadnje gledao (ViewedAt)

C) Zadnjih 5 rezervacija (Reservations)

- uzima se do 5 različitih listing-a koje je korisnik rezervisao (CreatedAt)
- listing-i koje je korisnik već rezervisao se **izbacuju iz preporuka** (exclude lista)

Cold start napomena: ako korisnik nema dovoljno signala (npr. nema historiju pregleda/pretrage), preporuke prelaze na fallback logiku Popular().

1.2. Kreiranje mape popularnosti (globalni signal)

Popularnost listing-a računa se na osnovu:

- broja pregleda u posljednjih **7 dana**
- broja rezervacija u posljednjih **30 dana**

Ove vrijednosti se pripremaju kao dvije mape/dictionary:

- views7d[ListingId] = broj pregleda
- res30d[ListingId] = broj rezervacija

1.3. Definisanje profila historije (sličnost)

Da bi sistem mogao procijeniti sličnost kandidata i korisnikovog ponašanja, iz historije se uzimaju profili listing-a:

- CityId
- RentTypeId
- PricePerNight
- RoomsCount

Ovi profili služe kao “reference” za scoring kandidata.

1.4. Kandidati za preporuku

Sistem prvo uzima bazu kandidata:

- Listings gdje je IsActive == true
- izbacuje listing-e koje korisnik već rezervisao (exclude)
- uzima **newest 250** listing-a (OrderByDescending(CreatedAt).Take(250))

Za svakog kandidata se dodatno učitava:

- City, RentType
- naslovna slika (cover) ili prva slika po SortOrder
- prosječna ocjena AvgRating
- broj recenzija ReviewsCount

1.5. Izračun ukupnog Score-a (glavna logika)

Za svakog kandidata računa se integer score koji se sastoji od 3 grupe signala:

A) Signal zadnje pretrage (Search boost)

Ako postoji lastSearch, score se povećava prema podudaranju:

- **Grad**: ako c.CityId == lastSearch.CityId → +40
- **Tip najma**: ako c.RentTypeId == lastSearch.RentTypeId → +25
- **Cijena**:
    - ako PricePerNight >= MinPrice → +10
    - ako PricePerNight <= MaxPrice → +10
- **Sobe**: ako RoomsCount == lastSearch.RoomsCount → +10
- **Gosti**: ako MaxGuests >= lastSearch.Guests → +10
- **Sort signal (mali boost)**:
    - priceasc → +2
    - pricedesc → +1
    - distanceasc i DistanceToCenterKm <= 2 → +2

B) Sličnost sa historijom (History similarity)

Za svaki listing iz historyListings kandidat dobija bodove:

- isti grad → +6
- isti rent type → +5
- cijena u opsegu ±20% u odnosu na historijski listing → +4
- isti broj soba → +2

(Pošto se petlja kroz više historijskih listing-a, kandidat može dobiti više bodova ako se podudara sa više elemenata iz historije.)

C) Popularnost i kvalitet (Popularity + Quality)

- pregledi u 7 dana: + min(10, views7d / 3)
- rezervacije u 30 dana: + min(15, reservations * 2)
- rating: + min(10, AvgRating * 2)
- broj recenzija: + min(5, ReviewsCount / 10)

1.6. Rangiranje i prikaz (Top-N)

Kandidati se sortiraju:

1. po Score opadajuće
2. zatim po CreatedAt opadajuće (novije ima prednost kod istog score-a)

Sistem vraća take rezultata (default 15, max 50).

Ako nakon rangiranja nema rezultata (ili nema signala) sistem aktivira fallback:

- return await Popular(take);

```
        // ☑ GET: api/listings/recommended?take=15
        [HttpGet("recommended")]
        [Authorize]
        0 references
        public async Task<IActionResult> Recommended([FromQuery] int take = 15)
        {
            take = Math.Clamp(take, 1, 50);

            var userId = GetUserIdOrNull();
            if (!userId.HasValue) return Unauthorized();

            // 1) Zadnja pretraga
            var lastSearch = await _db.SearchHistory
                .AsNoTracking()
                .Where(s => s.UserId == userId.Value)
                .OrderByDescending(s => s.SearchedAt)
                .FirstOrDefaultAsync();

            // 2) Zadnjih 10 views
            var recentViewedIds = await _db.ListingViews
                .AsNoTracking()
                .Where(v => v.UserId == userId.Value)
                .OrderByDescending(v => v.ViewedAt)
                .Select(v => v.ListingId)
                .Distinct()
                .Take(10)
                .ToListAsync();

            // 3) Zadnjih 5 rezervacija
            var recentReservedIds = await _db.Reservations
                .AsNoTracking()
                .Where(r => r.UserId == userId.Value)
                .OrderByDescending(r => r.CreatedAt)
```

96 % 0 1 Ln: 217 Ch: 33 SPC CRLF

```
            var recentReservedIds = await _db.Reservations
                .AsNoTracking()
                .Where(r => r.UserId == userId.Value)
                .OrderByDescending(r => r.CreatedAt)
                .Select(r => r.PropertyId)
                .Distinct()
                .Take(5)
                .ToListAsync();

            // Ne preporučuj ono što je user već rezervisao
            var exclude = recentReservedIds;

            // 4) Popularnost mape
            var fromViews = DateTime.UtcNow.AddDays(-7);
            var fromRes = DateTime.UtcNow.AddDays(-30);

            var views7d = await _db.ListingViews
                .AsNoTracking()
                .Where(v => v.ViewedAt >= fromViews)
                .GroupBy(v => v.ListingId)
                .Select(g => new { ListingId = g.Key, Cnt = g.Count() })
                .ToDictionaryAsync(x => x.ListingId, x => x.Cnt);

            var res30d = await _db.Reservations
                .AsNoTracking()
                .Where(r => r.CreatedAt >= fromRes)
                .GroupBy(r => r.PropertyId)
                .Select(g => new { ListingId = g.Key, Cnt = g.Count() })
                .ToDictionaryAsync(x => x.ListingId, x => x.Cnt);

            // 5) Listing profili iz historije (za "sličnost")
            var historyIds = recentViewedIds.Concat(recentReservedIds).Distinct().ToList();

            var historyListings = await _db.Listings
```

96 % 0 1 Ln: 217 Ch: 33 SPC CRLF

RentLoop.API | RentLoop.API.Controllers.ListingsController | Recommended(int take = 15)

```
10   namespace RentLoop.API.Controllers
14       public class ListingsController : ControllerBase
219          public async Task<IActionResult> Recommended([FromQuery] int take = 15)
277              var historyListings = await _db.Listings
278                  .AsNoTracking()
279                  .Where(l => historyIds.Contains(l.Id))
280                  .Select(l => new { l.Id, l.CityId, l.RentTypeId, l.PricePerNight, l.RoomsCount })
281                  .ToListAsync();
282
283              // 6) Kandidati (limit)
284              var baseQuery = _db.Listings
285                  .AsNoTracking()
286                  .Include(l => l.City)
287                  .Include(l => l.RentType)
288                  .Where(l => l.IsActive && !exclude.Contains(l.Id));
289
290              // ako ima zadnja pretraga, favorizuj taj grad (ali i dalje uzmi newest limit)
291              if (lastSearch?.CityId != null)
292              {
293                  // Ne filtriramo strogo samo taj grad da ne ispadne prazno, samo ostavljamo kandidatima šansu
294                  // Kandidati su newest 250, scoring će pogurati grad
295              }
296
297              var candidates = await baseQuery
298                  .OrderByDescending(l => l.CreatedAt)
299                  .Take(250)
300                  .Select(l => new
301                  {
302                      l.Id,
303                      l.Name,
304                      l.PricePerNight,
305                      City = l.City != null ? l.City.Name : "",
306                      RentType = l.RentType != null ? l.RentType.Name : "",
307                      l.CityId,
308                      l.RentTypeId,
309                      l.RoomsCount,
310                      l.MaxGuests,
```

96 % | 0 | 1 | Ln: 217 | Ch: 33 | SPC | CRLF

```
10   namespace RentLoop.API.Controllers
14       public class ListingsController : ControllerBase
219          public async Task<IActionResult> Recommended([FromQuery] int take = 15)
300                  .Select(l => new
304                      l.PricePerNight,
305                      City = l.City != null ? l.City.Name : "",
306                      RentType = l.RentType != null ? l.RentType.Name : "",
307                      l.CityId,
308                      l.RentTypeId,
309                      l.RoomsCount,
310                      l.MaxGuests,
311                      l.DistanceToCenterKm,
312                      l.CreatedAt,
313
314                      CoverUrl = _db.PropertyImages
315                          .Where(pi => pi.PropertyId == l.Id && pi.IsCover)
316                          .Select(pi => pi.Url)
317                          .FirstOrDefault()
318                          ?? _db.PropertyImages
319                              .Where(pi => pi.PropertyId == l.Id)
320                              .OrderBy(pi => pi.SortOrder)
321                              .Select(pi => pi.Url)
322                              .FirstOrDefault(),
323
324                      AvgRating = _db.Reviews
325                          .Where(rv => rv.PropertyId == l.Id)
326                          .Select(rv => (double?)rv.Rating)
327                          .Average() ?? 0,
328
329                      ReviewsCount = _db.Reviews.Count(rv => rv.PropertyId == l.Id)
330                  })
331                  .ToListAsync();
332
333              int Score(dynamic c)
334              {
335                  int score = 0;
336
```

```
337                 // A) Pretraga korisnika
338                 if (lastSearch != null)
339                 {
340                     if (lastSearch.CityId.HasValue && c.CityId == lastSearch.CityId.Value) score += 40;
341                     if (lastSearch.RentTypeId.HasValue && c.RentTypeId == lastSearch.RentTypeId.Value) score += 25;
342
343                     if (lastSearch.MinPrice.HasValue && c.PricePerNight >= lastSearch.MinPrice.Value) score += 10;
344                     if (lastSearch.MaxPrice.HasValue && c.PricePerNight <= lastSearch.MaxPrice.Value) score += 10;
345
346                     if (lastSearch.RoomsCount.HasValue && c.RoomsCount == lastSearch.RoomsCount.Value) score += 10;
347                     if (lastSearch.Guests.HasValue && c.MaxGuests >= lastSearch.Guests.Value) score += 10;
348
349                     // Sort signal (mali boost)
350                     if (!string.IsNullOrWhiteSpace(lastSearch.Sort))
351                     {
352                         var s = lastSearch.Sort!.ToLower();
353                         if (s == "priceasc") score += 2;      // korisnik voli niže cijene
354                         if (s == "pricedesc") score += 1;     // korisnik voli premium
355                         if (s == "distanceasc" && c.DistanceToCenterKm <= 2) score += 2;
356                     }
357                 }
358
359                 // B) Prethodne rezervacije / pregledi (sličnost)
360                 foreach (var h in historyListings)
361                 {
362                     if (c.CityId == h.CityId) score += 6;
363                     if (c.RentTypeId == h.RentTypeId) score += 5;
364
365                     var lower = h.PricePerNight * 0.8m;
366                     var upper = h.PricePerNight * 1.2m;
367                     if (c.PricePerNight >= lower && c.PricePerNight <= upper) score += 4;
368
369                     if (c.RoomsCount == h.RoomsCount) score += 2;
```

```
367                     if (c.PricePerNight >= lower && c.PricePerNight <= upper) score += 4;
368
369                     if (c.RoomsCount == h.RoomsCount) score += 2;
370                 }
371
372                 // C) Popularnost
373                 if (views7d.TryGetValue((int)c.Id, out var vCnt)) score += Math.Min(10, vCnt / 3);
374                 if (res30d.TryGetValue((int)c.Id, out var rCnt)) score += Math.Min(15, rCnt * 2);
375
376                 score += (int)Math.Min(10, c.AvgRating * 2);
377                 score += (int)Math.Min(5, c.ReviewsCount / 10);
378
379                 return score;
380             }
381
382             var ranked = candidates
383                 .Select(x => new
384                 {
385                     x.Id,
386                     x.Name,
387                     x.PricePerNight,
388                     x.City,
389                     x.RentType,
390                     x.RoomsCount,
391                     x.MaxGuests,
392                     x.DistanceToCenterKm,
393                     x.CoverUrl,
394                     x.AvgRating,
395                     x.ReviewsCount,
396                     Score = Score(x),
397                     x.CreatedAt
398                 })
```

96 % 0 1 Ln: 217 Ch: 33 SPC CRLF

```
                x.CreatedAt
            })
            .OrderByDescending(x => x.Score)
            .ThenByDescending(x => x.CreatedAt)
            .Take(take)
            .ToList();

        // fallback ako nema historije/signala
        if (ranked.Count == 0)
            return await Popular(take);

        return Ok(ranked);
    }

    // ☑ POST: api/listings/{id}/view
    [HttpPost("{id:int}/view")]
    [Authorize]
    0 references
    public async Task<IActionResult> LogView(int id)
    {
        var userId = GetUserIdOrNull();
        if (!userId.HasValue) return Unauthorized();

        var exists = await _db.Listings.AnyAsync(l => l.Id == id && l.IsActive);
        if (!exists) return NotFound("Listing not found.");

        _db.ListingViews.Add(new ListingView
        {
            UserId = userId.Value,
            ListingId = id,
            ViewedAt = DateTime.UtcNow
        });
```

96 % 0 1 Ln: 217 Ch: 33 SPC CRLF

```
        _db.ListingViews.Add(new ListingView
        {
            UserId = userId.Value,
            ListingId = id,
            ViewedAt = DateTime.UtcNow
        });

        await _db.SaveChangesAsync();
        return Ok(new { message = "View logged." });
    }
```

1.7. Cold Start – novi korisnici / korisnici bez historije

Za korisnike bez dovoljno interakcija (npr. nov nalog), sistem ne može pouzdano personalizovati. Tada se koristi fallback endpoint Popular, koji preporučuje listing-e na osnovu globalne popularnosti (views + rezervacije) i kvaliteta (rating + recenzije).

Putanja do source code-a (Recommended + Popular)

- RentLoop.API/Controllers/ListingsController.cs

---

2) Popularni listing-i (Popular)

Opis

Popular endpoint služi kao:

- fallback za Recommended kada nema signala
- zasebna lista “trending/popular” sadržaja

Popularnost se računa kao kombinacija:

- views zadnjih 7 dana (jači signal)
- rezervacije zadnjih 30 dana (najjači signal)

- prosječna ocjena
- broj recenzija

```
152                 l.Id,
153                 l.Name,
154                 l.PricePerNight,
155                 City = l.City != null ? l.City.Name : "",
156                 RentType = l.RentType != null ? l.RentType.Name : "",
157                 l.RoomsCount,
158                 l.MaxGuests,
159                 l.DistanceToCenterKm,
160                 l.CreatedAt,
161
162                 CoverUrl = _db.PropertyImages
163                     .Where(pi => pi.PropertyId == l.Id && pi.IsCover)
164                     .Select(pi => pi.Url)
165                     .FirstOrDefault()
166                     ?? _db.PropertyImages
167                         .Where(pi => pi.PropertyId == l.Id)
168                         .OrderBy(pi => pi.SortOrder)
169                         .Select(pi => pi.Url)
170                         .FirstOrDefault(),
171
172                 AvgRating = _db.Reviews
173                     .Where(rv => rv.PropertyId == l.Id)
174                     .Select(rv => (double?)rv.Rating)
175                     .Average() ?? 0,
176
177                 ReviewsCount = _db.Reviews.Count(rv => rv.PropertyId == l.Id)
178             })
179             .ToListAsync();
180
181         int PopularScore(dynamic x)
182         {
183             int score = 0;
184             if (views7d.TryGetValue((int)x.Id, out var vCnt)) score += Math.Min(15, vCnt / 2);
```

96 % 0 1 Ln: 217 Ch: 33 SPC CRLF

```
120     // GET: api/listings/popular?take=15
121     [HttpGet("popular")]
        1 reference
122     public async Task<IActionResult> Popular([FromQuery] int take = 15)
123     {
124         take = Math.Clamp(take, 1, 50);
125
126         var fromViews = DateTime.UtcNow.AddDays(-7);
127         var fromRes = DateTime.UtcNow.AddDays(-30);
128
129         var views7d = await _db.ListingViews
130             .AsNoTracking()
131             .Where(v => v.ViewedAt >= fromViews)
132             .GroupBy(v => v.ListingId)
133             .Select(g => new { ListingId = g.Key, Cnt = g.Count() })
134             .ToDictionaryAsync(x => x.ListingId, x => x.Cnt);
135
136         var res30d = await _db.Reservations
137             .AsNoTracking()
138             .Where(r => r.CreatedAt >= fromRes)
139             .GroupBy(r => r.PropertyId)
140             .Select(g => new { ListingId = g.Key, Cnt = g.Count() })
141             .ToDictionaryAsync(x => x.ListingId, x => x.Cnt);
142
143         var listings = await _db.Listings
144             .AsNoTracking()
145             .Include(l => l.City)
146             .Include(l => l.RentType)
147             .Where(l => l.IsActive)
148             .OrderByDescending(l => l.CreatedAt)
149             .Take(250)
150             .Select(l => new
151             {
152                 l.Id,
153                 l.Name,
```

```
185                 if (res30d.TryGetValue((int)x.Id, out var rCnt)) score += Math.Min(30, rCnt * 3);
186                 score += (int)Math.Min(10, x.AvgRating * 2);
187                 score += (int)Math.Min(5, x.ReviewsCount / 10);
188                 return score;
189             }
190
191             var ranked = listings
192                 .Select(x => new
193                 {
194                     x.Id,
195                     x.Name,
196                     x.PricePerNight,
197                     x.City,
198                     x.RentType,
199                     x.RoomsCount,
200                     x.MaxGuests,
201                     x.DistanceToCenterKm,
202                     x.CoverUrl,
203                     x.AvgRating,
204                     x.ReviewsCount,
205                     Score = PopularScore(x),
206                     x.CreatedAt
207                 })
208                 .OrderByDescending(x => x.Score)
209                 .ThenByDescending(x => x.CreatedAt)
210                 .Take(take)
211                 .ToList();
212
213             return Ok(ranked);
214         }
```

2.1. Kandidati

- uzima se newest 250 aktivnih listing-a (IsActive)
- za svakog kandidata se učitava cover slika, prosječna ocjena i broj recenzija

---

2.2. PopularScore formula

Za svaki listing:

- views u 7 dana: + min(15, views7d / 2)
- rezervacije u 30 dana: + min(30, reservations * 3)
- rating: + min(10, AvgRating * 2)
- recenzije: + min(5, ReviewsCount / 10)

Zatim se lista sortira po score-u, pa po CreatedAt, i vraća se Top-N.

---

3) Entiteti koji učestvuju u preporukama (modeli)

Recommender koristi sljedeće entitete iz baze:

- SearchHistory
  Polja: UserId, CityId, RentTypeId, MinPrice, MaxPrice, RoomsCount, Guests, Sort, SearchedAt
- ListingView
  Polja: UserId, ListingId, ViewedAt

- Reservation
  Polja: UserId, PropertyId, CreatedAt (+ ostala rezervaciona polja)
- Listing
  Polja: CityId, RentTypeId, PricePerNight, RoomsCount, MaxGuests, DistanceToCenterKm, IsActive, CreatedAt (+ ostalo)
- Review (za rating i count)
- PropertyImage (za cover sliku)